

LAB-03 Component Diagram

Academic Elective Bidding & Allocation System

Name: Pratham S Reddy

SRN: PES1UG24AM200

Class: AIML 5D

Date: 09/09/2026

Introduction:

Architecture Selection: We chose Microservices Architecture for the Academic Elective Bidding & Allocation System.

Why not Layered: Layered Architecture would be simpler, but it would make independent scaling of the allocation solver harder. The system has clearly separated responsibilities such as bidding, validation, allocation, administration, and result publication.

Why not Client-Server: Client-Server would centralize control, but it would create a stronger central bottleneck. The allocation workload can become much heavier than normal student interactions, so independently scaling the allocation service is more suitable.

Reason 1 - Separation of Responsibilities:

The system has clearly separable responsibilities: bidding and credit management, prerequisite and bid validation, elective allocation, academic administration, and result publication. These can operate as independently deployable services, making the system easier to modify without changing the whole application.

Reason 2 - Independent Scaling of Allocation:

The allocation workload can become much heavier than normal student interactions. The requirements specify processing 5,000 student bids and resolving schedule conflicts in under 30 seconds. The Allocation Service can therefore be scaled independently during peak bidding periods.

Security Advantage:

Microservices allow sensitive operations to be isolated behind service boundaries and controlled APIs. Authentication and authorization can be enforced at the access layer, while the bidding, validation, and administration services can apply role-specific access rules. This is useful because students and the Academic Registrar have different permissions.

Performance Benefit:

The Allocation Service can be deployed and scaled independently from student-facing services. During a large allocation run, additional allocation-service instances can process workload without unnecessarily replicating the Student Portal or administration components. This supports the under-30-second allocation target specified in the requirements.

Trade-off:

Microservices introduce network communication, service coordination, and data-consistency overhead. However, the benefits of independent scaling, fault isolation, and separation of the allocation workload outweigh the added complexity.

Requirements Traceability - FR/NFR to Architecture Mapping

ID	Type	Description	Priority	How the Architecture Addresses It
FR-001	Functional	The system shall allow students to distribute 100 bidding credits across their ranked elective preferences and validate prerequisite course completions.	High	Bidding Service manages credits and preferences, while Validation Service checks prerequisites before bids proceed.
FR-002	Functional	The system shall allow students to view available elective courses with course details and eligibility requirements before submitting bids.	High	Student Portal accesses academic data and services to display available electives and eligibility information.
FR-003	Functional	The system shall validate submitted bids and prevent bids that exceed available credits or violate prerequisite requirements.	High	Validation Service validates credit limits and prerequisites and returns validation status to the Bidding Service.
FR-004	Functional	The system shall allocate electives based on student bids while resolving allocation constraints and preventing timetable collisions.	High	Allocation Service runs the allocation solver, applies constraints, and checks timetable conflicts independently.

FR-005	Functional	The system shall allow students to view their final elective allocation results after the allocation process is completed.	Medium	Results Service stores and publishes allocation results, which are accessed by the Student Portal.
NFR-001	Performance and Security	The final elective allocation solver shall process 5,000 student bids and resolve schedule conflicts in under 30 seconds while maintaining required security standards.	High	Allocation Service can be scaled independently during peak allocation periods, while service boundaries and controlled APIs support security.
NFR-002	Reliability and Data Integrity	The system shall maintain accurate student bidding and allocation data throughout the process and prevent loss or corruption of submitted bids.	High	Academic Database provides persistent storage for student, course, prerequisite, bidding, timetable, and allocation data; services access it through controlled interfaces.

Component Modelling:

The component diagram represents the selected Microservices Architecture using UML component notation. It contains seven components and six interfaces.

Components:

Component	Responsibility
Student Portal	Login, view electives, submit/modify bids, view allocation results
Bidding Service	Manage bidding credits, store preferences, submit bids, enforce credit limits
Validation Service	Check prerequisites, validate credit limits and submitted bids, return validation status
Allocation Service	Run allocation solver, apply constraints, check timetable conflicts, generate allocations
Admin Service	Manage elective courses, configure bidding, support registrar controls
Academic Database	Store students and profiles, courses and prerequisites, bids and credits, timetables
Results Service	Store allocation results, publish results, provide student access

Interfaces:

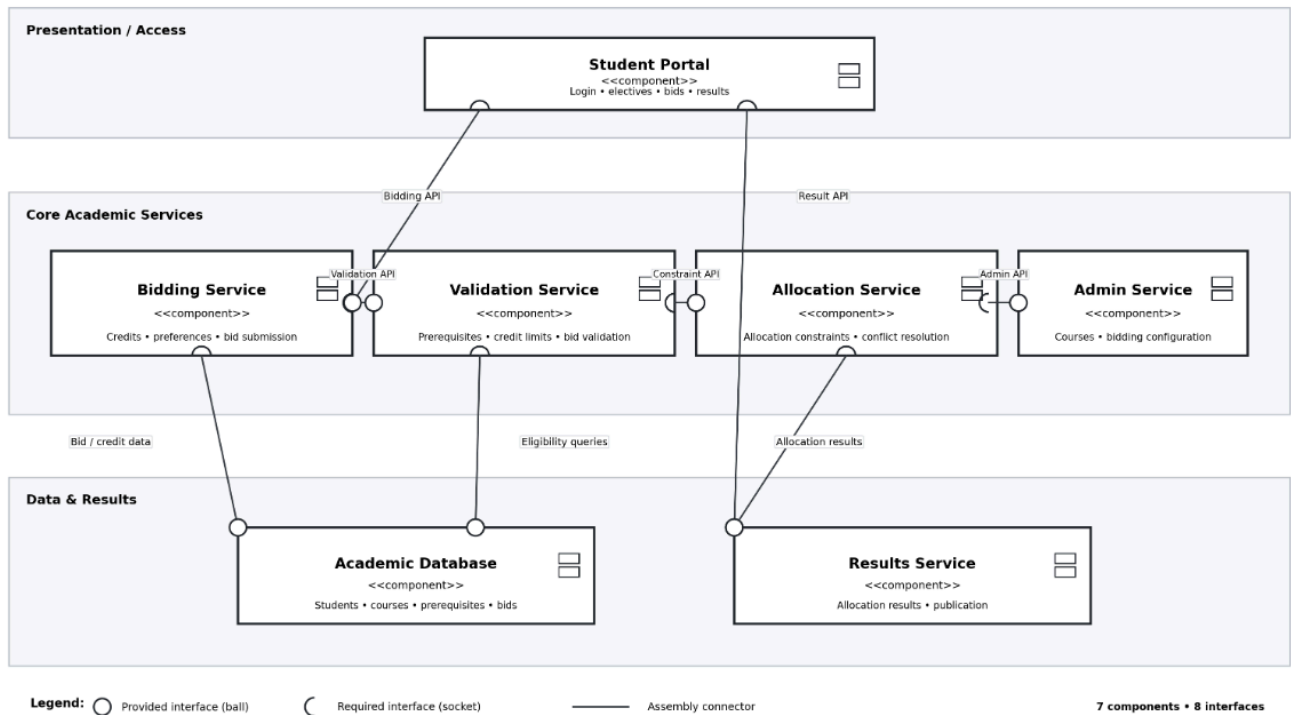
Bidding API, Validation API, Allocation API, Result API, Database Access Interface, and Result Access API connect the components through provided and required UML interfaces.

UML Notation:

A ball represents a provided interface, while the open socket / half-hemisphere represents a required interface. Solid connectors represent assembly or service communication.

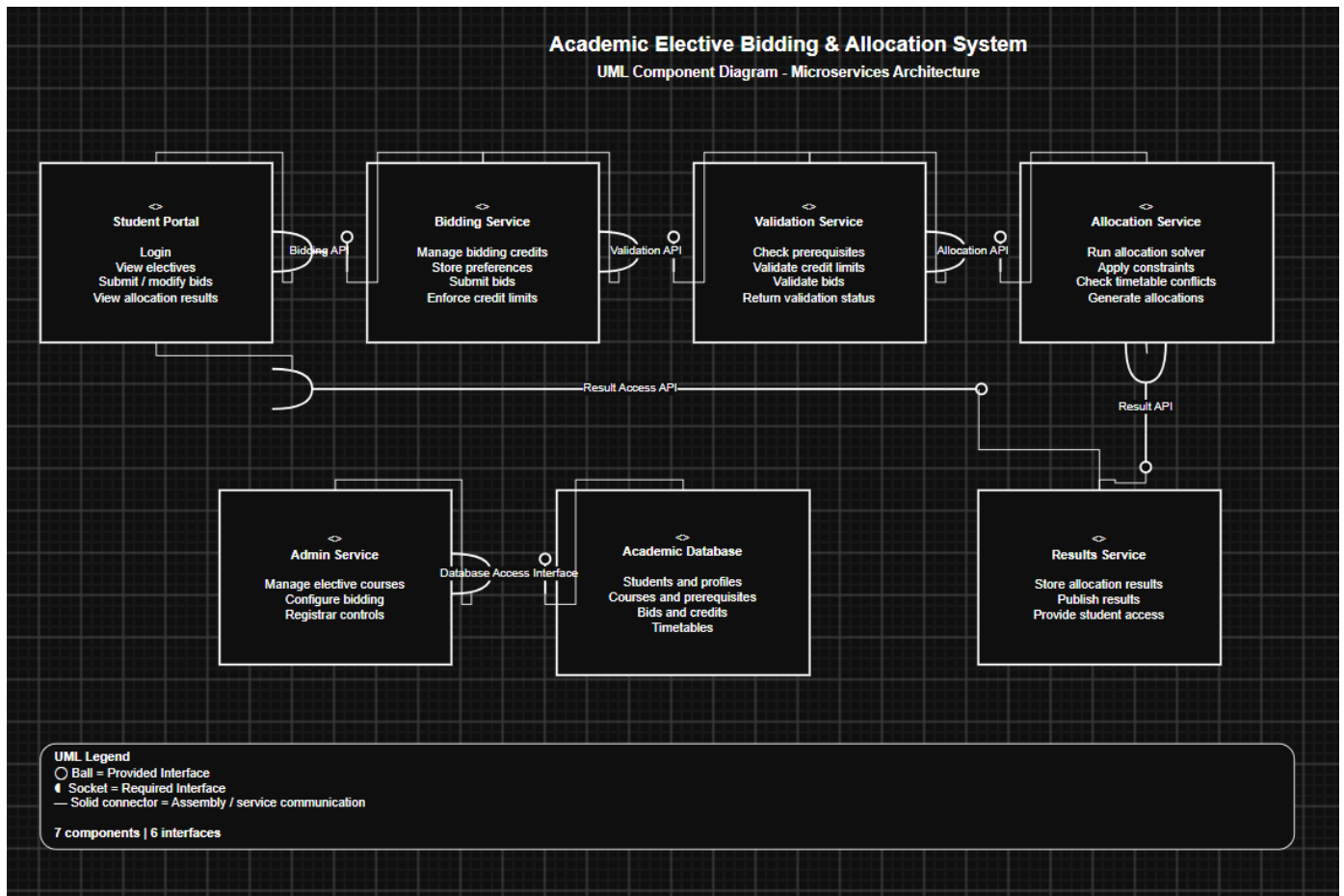
Academic Elective Bidding & Allocation System

UML Component Diagram — Microservices Architecture



Component Diagram:

Academic Elective Bidding & Allocation System - Microservices Architecture



Ball = Provided Interface | Socket = Required Interface | Solid connector = Assembly / service communication

7 components | 6 interfaces